```
'''
LP-III Assignment 2 (Group A - DAA)
Write a program to implement Huffman Encoding using a greedy strategy.
Name: Tanmay Sandip Bothara
Roll No: 22        Batch: BE_Comp_A1
'''

import heapq
# Node for Huffman Tree
class Node:
    def __init__(self, freq, char):
        self.freq = freq
        self.char = char
        self.left = None
        self.right = None
    # Compare nodes based on frequency
    def __lt__(self, other):
        return self.freq < other.freq

# Build Huffman Tree
def buildTree(characters, frequencies):
    heap = []
    # Create a node for each character
    for i in range(len(characters)):
        heapq.heappush(heap, Node(frequencies[i], characters[i]))
    # Combine nodes until only one node remains
    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        newNode = Node(left.freq + right.freq, None)
        newNode.left = left
```

```
        newNode.right = right
        heapq.heappush(heap, newNode)
    return heap[0]

# Generate Huffman Codes
def getCodes(root, code):
    if root is None:
        return
    # If it is a leaf node, print the code
    if root.char is not None:
        if code == "":
            code = "0"      # Only one character case
        print(root.char, ":", code)
        return
    getCodes(root.left, code + "0")
    getCodes(root.right, code + "1")



n = int(input("Enter the number of characters: "))
characters = []
frequencies = []
for i in range(n):
    ch = input("Enter character: ")
    freq = int(input("Enter frequency: "))
    characters.append(ch)
    frequencies.append(freq)
root = buildTree(characters, frequencies)
print("\nHuffman Codes:")
getCodes(root, "")

'''
```

OUTPUT:-

Enter the number of characters: 6

Enter character: a

Enter frequency: 5

Enter character: b

Enter frequency: 9

Enter character: c

Enter frequency: 12

Enter character: d

Enter frequency: 13

Enter character: e

Enter frequency: 16

Enter character: f

Enter frequency: 45

Huffman Codes:

f : 0

c : 100

d : 101

a : 1100

b : 1101

e : 111

'''